Encapsulation

CS300: Programming II

Cole Nelson

Midterm #1

Tonight at 5:45 pm. Show up 10-15 minutes early. Rooms and confirmations were emailed last Friday.

Nothing from today or Monday is on it.

Go over exam procedure.

Learning Objectives

1. Be able to **justify** declaring a field `private`.
2. Be able to **write** an accessor that does not leak a mutable field.
3. Be able to **write** a mutator that rejects invalid input.
4. Be able to **write** a `toString` method.

What could go wrong with `public` ?

```java
public class Book {
  public String title;
  public int daysLeft;
}
```

```java
dune.daysLeft = -500;
```

Nothing would stop this as they are `public`, but we can agree that it is not ideal. How do we stop this? `private`, but with accessors and mutators.

`private` allows for encapsulation

Encapsulation: the data and the methods that use it live in one class -- and nothing outside reaches that data except through those methods.

A `public` field is open to anyone, a `private` field (with `public` accessors and mutators) allow you to put up guard rails.

An accessor gets a value

```
public int getDaysLeft() {
  return this.daysLeft;
}
```

A **getter**. One line, no arguments, returns the field. Java's convention is `get` + the field name -- `isCheckedOut()` for a `boolean`.

A mutator changes a value -- on your own terms

```
public void setDaysLeft(int days) {
  this.daysLeft = days;
}
```

A **setter**. And written like that, it is exactly as dangerous as the public field was. `-500` still gets in.

Guard, then assign

```
public void setDaysLeft(int days) {
  if (days < 0) {
    throw new IllegalArgumentException("days cannot be negative, got " + days);
  }
  this.daysLeft = days;
}
```

This is why the field is private. The setter is a door, and now it has a lock on it.

Not every field needs a setter

A `Book` 's title never changes. Neither does its author.

Leave the setter out and the field is **read-only** to the outside world -- enforced by the compiler, not by a comment asking nicely.

Eclipse Exercise

TODO 1 write your getters and setters in `Book.java`.

Tophat Q1

A check on getters and setters.

→ Open Tophat to vote.

The getter that gave it away

```
private ArrayList<String> tags;

public ArrayList<String> getTags() {
  return this.tags;
}
```

```
dune.getTags().add("BANNED");
```

The field is `private`. No assignment, no `new`. Simply a getter, but... the `Book` changed anyway.

Hand out a copy

```
public ArrayList<String> getTags() {
  ArrayList<String> newTags = new ArrayList<String>();
  for(int i = 0; i < this.tags.size(); i++) {
    newTags.add(this.tags.get(i));
  }
  return newTags;
}
```

A new list (with the same contents). Whatever the caller does to it does not affect the actual list of tags.

Hand out a copy

```
public ArrayList<String> getTags() {
  return new ArrayList<String>(this.tags);
}
```

Simpler, built-in way to copy an `ArrayList`.

Eclipse Exercise

Stop the accessor leaking. **TODO 2** in `Book.java`.

`toString` lets the object introduce itself

```
public String toString() {
  // Note: Don't sysout here! Let the caller take care of that.
  return this.title + " by " + this.author;
}
```

```
System.out.println(dune.toString());     // either work!
System.out.println(dune);                // implicit toString()
```

Without one you get `Book@6d06d69c`.

Should be annotated with `@Override`, a topic for next week's classes on inheritance.

Eclipse Exercise

Write `toString`. **TODO 3** in `Book.java`.

Tophat Q2

A check on today's topic.

→ Open Tophat to vote.

Questions?